

PART

III

서울시 인구
행정동별 분석
(Python)





III | 서울시 인구 행정동별 분석

SGIS 자료제공 메뉴에서 내려받은 서울시 행정동 경계(`bnd_dong_11_2024_2Q.shp`), 2023년 서울시 인구 통계(`11_2023년_인구총괄(총인구).csv`)를 파이썬(Jupyter Notebook)으로 분석하는 과정을 설명합니다.

1 분석 라이브러리 로드 및 사용 준비하기

- ▶ geopandas, pandas, matplotlib 라이브러리를 임포트합니다. 각각 공간 데이터 분석, 표 형식 데이터 처리, 시각화에 사용됩니다.

```
[1]: import geopandas as gpd # 지리정보 파일 읽고 처리
import pandas as pd # 표 형식 데이터 처리
import matplotlib.pyplot as plt # 그래프, 지도 등 시각화
```

```
plt.rcParams['font.family'] = 'Malgun Gothic' # 윈도우에서 한글 폰트 설정
```

그래프, 지도 등 시각화에 사용되는 matplotlib는 사용하기 전 한글 폰트를 명시적으로 설정해야 합니다. 한글이 사용되는 경우, 폰트 파일 로딩 지연 문제 등으로 소스코드 첫부분에서 실행해줍니다.

주의

지리정보를 처리하는데 사용되는 geopandas 라이브러리는 별도로 설치해주어야 합니다. 윈도우 앱 목록에서 **Anaconda** → **Anaconda Prompt** 를 선택하고 아래의 명령어로 geopandas를 설치합니다.

```
Anaconda Prompt
(base) C:\> pip install geopandas
```

2 서울시 경계 및 통계 데이터 불러오기

- ▶ gpd.read_file()로 경계 데이터를 불러오고, pd.read_csv()로 인구통계 데이터를 불러옵니다. 두 파일을 각각 bord_sido와 stat에 저장합니다. (※ 경로는 사용자 PC에 맞게 수정)

```
[2]: prj_dir = 'C:/SGIS/Python/서울시 행정동 인구 분석'
bord_sido = gpd.read_file(prj_dir + '/' + 'bnd_dong_11_2024_2Q.shp')
stat = pd.read_csv(prj_dir + '/' + '11_2023년_인구총괄(총인구).csv',
names=['BASE_YEAR', 'ADM_CD', 'STAT_CD', 'POP'])
```

※ 통계 파일을 열어보면 구분자는 ','(콤마)이고, 첫 행에 컬럼명이 없는 것을 확인할 수 있습니다. 이 경우 파일을 불러오면서 컬럼명을 함께 지정할 수 있습니다.

3 데이터 미리보기

- ▶ head() 함수로 두 데이터의 첫 5개 행을 출력하여 구조를 확인합니다.

```
[3]: print(bord_sido.head()) # 기준날짜(BASE_DATE), 행정구역 이름(ADM_NM),
# 행정구역 코드(ADM_CD), 지리정보(geometry)로 구성
```

	BASE_DATE	ADM_NM	ADM_CD	\
0	20240630	잠실4동	11240750	
1	20240630	잠실7동	11240780	
2	20240630	잠실2동	11240790	
3	20240630	잠실6동	11240770	
4	20240630	방이2동	11240580	

	geometry
0	POLYGON ((966144.947 1947247.39, 966140.973 19...
1	POLYGON ((962306.123 1944876.429, 962298.368 1...
2	POLYGON ((963373.434 1945897.333, 963372.972 1...
3	POLYGON ((964764.987 1947265.422, 964790.647 1...
4	POLYGON ((965792.751 1946532.439, 965878.771 1...

```
[4]: print(stat.head()) # 기준년도(BASE_YEAR), 행정구역 코드(ADM_CD),
# 통계 코드(STAT_CD), 인구수(POP)
```

	BASE_YEAR	ADM_CD	STAT_CD	POP
0	2023	11010530	to_in_001	8377
1	2023	11010540	to_in_001	2115
2	2023	11010550	to_in_001	9136
3	2023	11010560	to_in_001	16475
4	2023	11010570	to_in_001	7557

4 통계 항목 필터링

- ▶ STAT_CD 컬럼에 있는 고유값을 확인하고 분석에 필요한 총인구('to_in_001')만 필터링하여 stat_total에 저장합니다.

```
[5]: # STAT_CD 열에 있는 고유 값 확인
unique_stat_cd = stat['STAT_CD'].unique()
print(unique_stat_cd)
```

```
['to_in_001' 'to_in_007' 'to_in_008']
```

```
[6]: # 총인구('to_in_001') 데이터만 필터링
stat_total = stat[stat['STAT_CD'] == 'to_in_001']
# 필터링 후 STAT_CD 고유값 재확인
unique_stat_cd = stat_total['STAT_CD'].unique()
print(unique_stat_cd)
```

```
['to_in_001']
```

5 분석에 필요한 컬럼만 선택하기

- ▶ stat_total에서 ADM_CD와 POP 컬럼만 남겨서 나머지 불필요한 열을 제거합니다.(추후 경계 파일과 병합 과정에서 필요한 컬럼만 저장)

```
[7]: stat_total = stat_total[['ADM_CD', 'POP']]
```

6 경계와 통계 데이터 타입 확인 및 변환하기

- ▶ 각각의 파일에 키값으로 사용가능한 행정구역 코드(ADM_CD) 데이터 타입 확인 후 문자열로 일치 시킵니다.(병합 시 발생하는 오류 방지)

```
[8]: print(bord_sido.dtypes) # 경계(bord_sido) 각 열의 데이터 타입 확인
```

```
BASE_DATE      object
ADM_NM         object
ADM_CD         object
geometry       geometry
dtype: object
```

```
[9]: print(stat_total.dtypes) # 통계(stat_total) 각 열의 데이터 타입 확인
```

```
ADM_CD      int64
POP         int64
dtype: object
```

```
[10]: stat_total['ADM_CD'] = stat_total['ADM_CD'].astype(str) # 문자열로 변환
```

7 경계와 통계 병합하기

- ▶ bord_sido와 stat_total 두 개의 데이터프레임을 'ADM_CD'를 기준으로 left join(왼쪽 조인) 하여 병합합니다.

```
[11]: bord_sido = bord_sido.merge(stat_total, on='ADM_CD', how='left')
print(bord_sido.isna().sum()) # 결측값이 있는지 확인
print(bord_sido.head()) # 병합 데이터 미리보기
```

```
BASE_DATE    0
ADM_NM       0
ADM_CD       0
geometry     0
POP          0
dtype: int64
```

	BASE_DATE	ADM_NM	ADM_CD	\	geometry	POP
0	20240630	잠실4동	11240750			19916
1	20240630	잠실7동	11240780			8581
2	20240630	잠실2동	11240790			32990
3	20240630	잠실6동	11240770			15381
4	20240630	방이2동	11240580			25888

8 총인구수를 기준으로 구간 나누기

- ▶ 총인구수('POP')를 5개의 구간으로 나누고 각 구간에 대한 라벨을 지정합니다.

```
[12]: print(bord_sido['POP'].min(), bord_sido['POP'].max()) # 최소값과 최대값 확인하기
```

26 52789

```
[13]: bins = [25, 10000, 20000, 30000, 40000, 53000] # 적절한 구간 지정
labels = ['25~10,000', '10,000~20,000', '20,000~30,000', '3,0000~40,000', '40,000~53,000'] # 구간의 라벨
bord_sido['POP_BINS'] = pd.cut(bord_sido['POP'], bins=bins, labels=labels, right=False)
print(bord_sido.head()) # 인구 구간('POP_BINS') 컬럼 추가 확인
```

	BASE_DATE	ADM_NM	ADM_CD	\	geometry	POP	POP_BINS
0	20240630	잠실4동	11240750			19916	10,000~20,000
1	20240630	잠실7동	11240780			8581	25~10,000
2	20240630	잠실2동	11240790			32990	3,0000~40,000
3	20240630	잠실6동	11240770			15381	10,000~20,000
4	20240630	방이2동	11240580			25888	20,000~30,000

9 지도 시각화 하기

```
[14]: # 시각화 설정(그림과 축 설정)
fig, ax = plt.subplots(1, 1, figsize=(12, 12))

# 지도 시각화('POP_BINS' 컬럼을 기준으로 OrRd 색상 맵 사용)
bord_sido.plot(column='POP_BINS', ax=ax, legend=True, cmap='OrRd')

# 서울시 행정동 경계 추가 (경계선만 보이도록 설정, 배경은 투명)
bord_sido.plot(ax=ax, color='none', edgecolor='black', linewidth=0.5)

# (추가) 서울시 시군구 경계를 추가하여 명확하게 분석하기
bord_sgg = gpd.read_file(prj_dir + '/' + 'bnd_sigungu_11_2024_2Q.shp')
bord_sgg.plot(ax=ax, color='none', edgecolor='black', linewidth=2)
bord_sgg['centroid'] = bord_sgg.geometry.centroid # 중심점 계산
for idx, row in bord_sgg.iterrows():
    ax.text(row['centroid'].x, row['centroid'].y, row['SIGUNGU_NM'], fontsize=15, ha='center',
            color='black', bbox=dict(facecolor='white', alpha=0.7, edgecolor='none',
            boxstyle='round,pad=0.1'))

# 범례 위치 및 제목 설정
ax.get_legend().set_bbox_to_anchor((0.5, -0.05)) # 범례 위치 조정
ax.get_legend().set_title("<인구 범위>") # 범례 제목 설정
ax.set_title("<서울시 인구 행정동별 분석>", fontsize=24) # 지도 제목 설정

# 레이아웃 조정 및 출력
plt.tight_layout() # 그래프 레이아웃이 겹치지 않도록 자동 조정
plt.savefig(prj_dir + '/' + '서울시 인구 행정동별 분석 지도.png')
plt.show() # 시각화 결과 화면에 출력
```

| 서울시 인구 행정동별 분석 |

